

CSCI 3202 Project Intermediate Report

Maxwell Rodgers

Project Overview

In this project I am making an agent to play the abstract strategy game Tak. In Tak, players take turns placing or moving their pieces on a square board (usually between 3×3 and 8×8). The goal is to make a 'road' of orthogonally adjacent pieces from any side of the board to the opposite side, similar to Hex. The game can also end when a player runs out of pieces or there are no empty spaces left on the board. In this case, the player who controls the most squares wins. There is a handicap for the first player in the form of a 'komi' (taken from the game Go). The komi is a value that is added to the count of the 2nd player's stones (this does not matter for games that end in a road). The game can be a tie if the komi is a whole number. Full rules can be found at ustak.org.

Current Progress

So far I have implemented everything surrounding the game itself, a pretty printer for the board state, a random player and a human player interface. For the game itself, all rules and mechanics are implemented. The `Game` class has a `play()` method that has each player take a move until the game is over. The `Player` class has 3 classes that inherit it: `HumanPlayer`, `RandomPlayer` and `MinimaxPlayer`. Each of these has a `get_move(game, opener)` method that takes in the game state and gets a move using their respective method. `RandomPlayer` uses the `enumerate_moves(player, opener)` function that takes in a player as an argument and returns all the legal moves for that player, and chooses one at random.

Future Plans

- I have tested each part manually to ensure that it works in most scenarios but am planning on writing test cases for some parts to make sure.
- I am going to implement the minimax player and create an alpha-beta player
- I will probably end up improving the performance. I have already done a few things such as caching possible drops, but want to switch to a loop based DFS and look at what else I can do to improve the hottest functions.

How to Run

Everything should be set up so that you can just run the included `main.py` file. You should be prompted for a number of options relating to the game. I recommend using two random players to see a full game. The pieces use ANSI terminal color to indicate the player (the background is the color of the piece). The stones are lettered based on their type (Flatstone, Standing stone, Capstone). If you choose to try the human player interface, be warned that there is a very limited amount of input validation so it is likely that if you don't know the game very well (or honestly even if you do) that it will throw an exception.

Testing with Random Players

I tested the program with 1000 runs of random player vs random player for each combination of board size and komi. The full results are available below, but in summary, there is a slight first player advantage. With random players, this advantage is equalized with less than 0.5 komi. I suspect this is because oftentimes games with random players will end without a road (due to them placing a lot of pieces) so the komi is much more significant than it would be with real players. I also did 100,000 trials of random games on a 3x3 board with 0 komi that supports this, with 48% of games being a win for P1, 43% being a win for P2 and 9% being ties.

Full Test Results:

test results:

100000 trials of random vs random with
komi 0 and board size 3 : P1: 48145, P2:
42865, Ties: 8990

1000 games, 3x3 board, 0.0 komi:
P1: 463, P2: 431, Ties: 106
1000 games, 3x3 board, 0.5 komi:
P1: 471, P2: 529, Ties: 0
1000 games, 3x3 board, 1.0 komi:
P1: 406, P2: 527, Ties: 67
1000 games, 3x3 board, 1.5 komi:
P1: 423, P2: 577, Ties: 0
1000 games, 3x3 board, 2.0 komi:
P1: 357, P2: 603, Ties: 40
1000 games, 3x3 board, 2.5 komi:
P1: 363, P2: 637, Ties: 0
1000 games, 3x3 board, 3.0 komi:
P1: 359, P2: 626, Ties: 15

1000 games, 4x4 board, 0.0 komi:
P1: 426, P2: 431, Ties: 143
1000 games, 4x4 board, 0.5 komi:
P1: 431, P2: 569, Ties: 0
1000 games, 4x4 board, 1.0 komi:
P1: 309, P2: 576, Ties: 115
1000 games, 4x4 board, 1.5 komi:
P1: 305, P2: 695, Ties: 0
1000 games, 4x4 board, 2.0 komi:
P1: 254, P2: 654, Ties: 92
1000 games, 4x4 board, 2.5 komi:
P1: 257, P2: 743, Ties: 0
1000 games, 4x4 board, 3.0 komi:
P1: 210, P2: 751, Ties: 39

1000 games, 5x5 board, 0.0 komi:
P1: 459, P2: 439, Ties: 102
1000 games, 5x5 board, 0.5 komi:
P1: 445, P2: 555, Ties: 0
1000 games, 5x5 board, 1.0 komi:
P1: 404, P2: 509, Ties: 87
1000 games, 5x5 board, 1.5 komi:
P1: 374, P2: 626, Ties: 0
1000 games, 5x5 board, 2.0 komi:
P1: 287, P2: 633, Ties: 80
1000 games, 5x5 board, 2.5 komi:
P1: 312, P2: 688, Ties: 0
1000 games, 5x5 board, 3.0 komi:
P1: 269, P2: 699, Ties: 32

1000 games, 6x6 board, 0.0 komi:
P1: 433, P2: 441, Ties: 126
1000 games, 6x6 board, 0.5 komi:
P1: 438, P2: 562, Ties: 0
1000 games, 6x6 board, 1.0 komi:
P1: 360, P2: 545, Ties: 95
1000 games, 6x6 board, 1.5 komi:
P1: 340, P2: 660, Ties: 0
1000 games, 6x6 board, 2.0 komi:
P1: 225, P2: 688, Ties: 87
1000 games, 6x6 board, 2.5 komi:
P1: 245, P2: 755, Ties: 0
1000 games, 6x6 board, 3.0 komi:
P1: 158, P2: 770, Ties: 72

1000 games, 7x7 board, 0.0 komi:
P1: 457, P2: 447, Ties: 96
1000 games, 7x7 board, 0.5 komi:
P1: 454, P2: 546, Ties: 0
1000 games, 7x7 board, 1.0 komi:
P1: 385, P2: 529, Ties: 86
1000 games, 7x7 board, 1.5 komi:
P1: 358, P2: 642, Ties: 0
1000 games, 7x7 board, 2.0 komi:
P1: 273, P2: 636, Ties: 91
1000 games, 7x7 board, 2.5 komi:
P1: 253, P2: 747, Ties: 0
1000 games, 7x7 board, 3.0 komi:
P1: 192, P2: 740, Ties: 68

1000 games, 8x8 board, 0.0 komi:
P1: 463, P2: 444, Ties: 93
1000 games, 8x8 board, 0.5 komi:
P1: 459, P2: 541, Ties: 0
1000 games, 8x8 board, 1.0 komi:
P1: 379, P2: 541, Ties: 80
1000 games, 8x8 board, 1.5 komi:
P1: 368, P2: 632, Ties: 0
1000 games, 8x8 board, 2.0 komi:
P1: 295, P2: 621, Ties: 84
1000 games, 8x8 board, 2.5 komi:
P1: 269, P2: 731, Ties: 0
1000 games, 8x8 board, 3.0 komi:
P1: 188, P2: 726, Ties: 86